

Sistemas Operacionais

Sincronização de Processos

```
class Conta {  
    int saldo = 100;  
  
    public void sacar(int valor) {  
        saldo -= valor;  
    }  
  
    public int getSaldo() {  
        return saldo;  
    }  
}
```

```
class Usuario extends Thread {
    Conta conta;

    public Usuario(Conta conta) {
        this.conta = conta;
    }

    public void run() {
        for (int i = 0; i < 10; i++) {
            conta.sacar(10);
            System.out.println(Thread.currentThread().getName() + " saldo: " + conta.saldo);
        }
    }
}
```

```
public class Principal {  
    public static void main(String[] args) {  
        Conta conta = new Conta();  
  
        Usuario u1 = new Usuario(conta);  
        Usuario u2 = new Usuario(conta);  
  
        u1.start();  
        u2.start();  
    }  
}
```

Resolvendo o problema

```
class Conta {  
    private int saldo = 100;  
  
    public synchronized void sacar(int valor) {  
        saldo -= valor;  
    }  
  
    public synchronized int getSaldo() {  
        return saldo;  
    }  
}
```

Resolvendo o problema

```
class Usuario extends Thread {
    Conta conta;

    public Usuario(Conta conta) {
        this.conta = conta;
    }

    public void run() {
        for (int i = 0; i < 10; i++) {
            conta.sacar(10);
            System.out.println(Thread.currentThread().getName() + " saldo: " + conta.getSaldo());
        }
    }
}
```

Produtor-Consumidor

Produtor-Consumidor

- O padrão produtor-consumidor é uma solução comum para o problema de coordenação entre threads que produzem dados e threads que consomem esses dados.
- Em Java, isso geralmente é implementado usando a sincronização inerente da linguagem, com métodos `synchronized`, `wait()` e `notify()` ou `notifyAll()`.

Produtor-Consumidor

```
3 public class Buffer {
4     private int conteudo;
5     private boolean disponivel = false;
6
7     public synchronized void set(int valor) {
8         while (disponivel) {
9             try {
10                 wait(); // Espera até o buffer estar vazio
11             } catch (InterruptedException e) { }
12         }
13         conteudo = valor;
14         disponivel = true;
15         notifyAll(); // Notifica que o buffer agora está cheio
16     }
17
18     public synchronized int get() {
19         while (!disponivel) {
20             try {
21                 wait(); // Espera até o buffer estar cheio
22             } catch (InterruptedException e) { }
23         }
24         disponivel = false;
25         notifyAll(); // Notifica que o buffer agora está vazio
26         return conteudo;
27     }
28 }
```

Produtor-Consumidor

```
3 public class Consumidor implements Runnable {
4     private Buffer buffer;
5
6     public Consumidor(Buffer buffer) {
7         this.buffer = buffer;
8     }
9
10    public void run() {
11        for (int i = 0; i < 10; i++) {
12            int valor = buffer.get();
13            System.out.println("Consumido: " + valor);
14        }
15    }
16 }
```

Produtor-Consumidor

```
3 public class Produtor implements Runnable {
4     private Buffer buffer;
5
6     public Produtor(Buffer buffer) {
7         this.buffer = buffer;
8     }
9
10    public void run() {
11        for (int i = 0; i < 10; i++) {
12            buffer.set(i);
13            System.out.println("Produzido: " + i);
14        }
15    }
16 }
```

Produtor-Consumidor

```
3 public class Principal {
4     public static void main(String[] args) {
5         Buffer bufferCompartilhado = new Buffer();
6         Thread produtorThread = new Thread(new Produtor(bufferCompartilhado));
7         Thread consumidorThread = new Thread(new Consumidor(bufferCompartilhado));
8
9         produtorThread.start();
10        consumidorThread.start();
11    }
12 }
```

Monitores

Monitores

- Monitores são mecanismos de sincronização de alto nível usados em programação concorrente para evitar condições de corrida quando múltiplos processos ou threads acessam recursos compartilhados.
- Eles encapsulam as variáveis compartilhadas ou recursos e os procedimentos que operam sobre esses recursos, garantindo que apenas um thread execute dentro do monitor em um dado momento.

Monitores

- Como funcionam os monitores:
 - Exclusão Mútua: Garante que apenas um thread possa executar uma função do monitor por vez. Isso é fundamental para proteger a integridade dos dados compartilhados.
 - Variáveis de condição: Usadas dentro do monitor para suspender threads até que alguma condição seja verdadeira. Isso é útil para controlar o acesso e a ordem dos threads que precisam de recursos compartilhados.

Monitores

- Suponha que temos um buffer compartilhado onde um produtor coloca dados e um consumidor os retira.
- O monitor seria usado para sincronizar o acesso ao buffer, garantindo que o produtor não adicione dados quando o buffer estiver cheio, e que o consumidor não tente remover dados quando o buffer estiver vazio.

Monitores

```
3 public class BufferCircular {
4     private final int[] buffer;
5     private int front = 0; // Indica a posição do próximo item a ser consumido
6     private int rear = 0;  // Indica a posição do próximo item a ser produzido
7     private int count = 0; // Número de itens no buffer
8
9     public BufferCircular(int size) {
10         buffer = new int[size];
11     }
12
13     public synchronized void put(int value) throws InterruptedException {
14         while (count == buffer.length) {
15             wait(); // Buffer cheio, espera até o consumidor consumir algum item
16         }
17         buffer[rear] = value;
18         rear = (rear + 1) % buffer.length;
19         count++;
20         notifyAll(); // Notifica as threads que estão esperando para consumir
21     }
22
23     public synchronized int get() throws InterruptedException {
24         while (count == 0) {
25             wait(); // Buffer vazio, espera até o produtor produzir algum item
26         }
27         int result = buffer[front];
28         front = (front + 1) % buffer.length;
29         count--;
30         notifyAll(); // Notifica as threads que estão esperando para produzir
31         return result;
32     }
33 }
```

```
3 public class BufferCircular {
4     private final int[] buffer;
5     private int front = 0; // Indica a posição do próximo item a ser consumido
6     private int rear = 0;  // Indica a posição do próximo item a ser produzido
7     private int count = 0; // Número de itens no buffer
8
9     public BufferCircular(int size) {
10         buffer = new int[size];
11     }
12
13     public synchronized void put(int value) throws InterruptedException {
14         while (count == buffer.length) {
15             wait(); // Buffer cheio, espera até o consumidor consumir algum item
16         }
17         buffer[rear] = value;
18         rear = (rear + 1) % buffer.length;
19         count++;
20         notifyAll(); // Notifica as threads que estão esperando para consumir
21     }
22
23     public synchronized int get() throws InterruptedException {
24         while (count == 0) {
25             wait(); // Buffer vazio, espera até o produtor produzir algum item
26         }
27         int result = buffer[front];
28         front = (front + 1) % buffer.length;
29         count--;
30         notifyAll(); // Notifica as threads que estão esperando para produzir
31         return result;
32     }
33 }
```

Monitores

```
3 class Produtor implements Runnable {
4     private BufferCircular buffer;
5
6     public Produtor(BufferCircular buffer) {
7         this.buffer = buffer;
8     }
9
10    public void run() {
11        try {
12            for (int i = 0; i < 10; i++) {
13                buffer.put(i);
14                System.out.println("Produzido: " + i);
15            }
16        } catch (InterruptedException e) {
17            Thread.currentThread().interrupt();
18        }
19    }
20 }
```

Monitores

```
3 class Consumidor implements Runnable {
4     private BufferCircular buffer;
5
6     public Consumidor(BufferCircular buffer) {
7         this.buffer = buffer;
8     }
9
10    public void run() {
11        try {
12            for (int i = 0; i < 10; i++) {
13                int valor = buffer.get();
14                System.out.println("Consumido: " + valor);
15            }
16        } catch (InterruptedException e) {
17            Thread.currentThread().interrupt();
18        }
19    }
20 }
```

Monitores

```
3 public class Principal {  
4     public static void main(String[] args) {  
5         BufferCircular buffer = new BufferCircular(5);  
6         Thread threadProdutor = new Thread(new Produtor(buffer));  
7         Thread threadConsumidor = new Thread(new Consumidor(buffer));  
8  
9         threadProdutor.start();  
10        threadConsumidor.start();  
11    }  
12 }
```

Semáforos

Semáforos

- Semáforos são uma ferramenta de sincronização utilizada para controlar o acesso a recursos comuns em ambientes de programação concorrente.
- Eles podem ser binários (semelhante a um mutex) ou contar um número específico de permissões que definem quantas threads podem acessar um recurso simultaneamente.

Semáforos

- Um semáforo mantém um conjunto de permissões:
- Acquire (adquirir): uma thread tenta obter uma permissão. Se nenhuma permissão estiver disponível, a thread é bloqueada até que uma permissão seja liberada por outra thread.
- Release (liberar): uma thread libera uma permissão, tornando-a disponível para outra thread que possa estar esperando.

Semáforos

```
7 public class Buffer {
8     private Queue<Integer> queue = new LinkedList<>();
9     private static final int CAPACITY = 5;
10    private Semaphore semFull = new Semaphore(0);
11    private Semaphore semEmpty = new Semaphore(CAPACITY);
12
13    public void put(int item) throws InterruptedException {
14        semEmpty.acquire(); // Adquire um espaço vazio
15        synchronized (this) {
16            queue.add(item);
17            System.out.println("Produzido: " + item);
18        }
19        semFull.release(); // Libera um espaço cheio
20    }
21
22    public int get() throws InterruptedException {
23        semFull.acquire(); // Adquire um espaço cheio
24        int item;
25        synchronized (this) {
26            item = queue.poll();
27            System.out.println("Consumido: " + item);
28        }
29        semEmpty.release(); // Libera um espaço vazio
30        return item;
31    }
32 }
```

```
7 public class Buffer {
8     private Queue<Integer> queue = new LinkedList<>();
9     private static final int CAPACITY = 5;
10    private Semaphore semFull = new Semaphore(0);
11    private Semaphore semEmpty = new Semaphore(CAPACITY);
12
13    public void put(int item) throws InterruptedException {
14        semEmpty.acquire(); // Adquire um espaço vazio
15        synchronized (this) {
16            queue.add(item);
17            System.out.println("Produzido: " + item);
18        }
19        semFull.release(); // Libera um espaço cheio
20    }
21
22    public int get() throws InterruptedException {
23        semFull.acquire(); // Adquire um espaço cheio
24        int item;
25        synchronized (this) {
26            item = queue.poll();
27            System.out.println("Consumido: " + item);
28        }
29        semEmpty.release(); // Libera um espaço vazio
30        return item;
31    }
32 }
```

Semáforos

```
3 public class Consumidor implements Runnable {
4     private Buffer buffer;
5
6     public Consumidor(Buffer buffer) {
7         this.buffer = buffer;
8     }
9
10    @Override
11    public void run() {
12        try {
13            for (int i = 0; i < 10; i++) {
14                buffer.get();
15                Thread.sleep(1000); // Simula tempo de consumo
16            }
17        } catch (InterruptedException e) {
18            Thread.currentThread().interrupt();
19        }
20    }
21 }
```

Semáforos

```
3 public class Produtor implements Runnable {
4     private Buffer buffer;
5
6     public Produtor(Buffer buffer) {
7         this.buffer = buffer;
8     }
9
10    @Override
11    public void run() {
12        try {
13            for (int i = 0; i < 10; i++) {
14                buffer.put(i);
15                Thread.sleep(1000); // Simula tempo de produção
16            }
17        } catch (InterruptedException e) {
18            Thread.currentThread().interrupt();
19        }
20    }
21 }
```

Semáforos

```
3 public class Principal {  
4     public static void main(String[] args) {  
5         Buffer bufferCompartilhado = new Buffer();  
6         Thread produtorThread = new Thread(new Produtor(bufferCompartilhado));  
7         Thread consumidorThread = new Thread(new Consumidor(bufferCompartilhado));  
8  
9         produtorThread.start();  
10        consumidorThread.start();  
11    }  
12 }
```

“Evolução natural” do problema produtor-consumidor

- Limitações da sincronização manual
- Precisamos controlar:
 - `wait()`
 - `notify()`
 - Semáforos
 - buffer cheio/vazio
 - exclusão mútua
 - O código fica mais complexo
 - Maior chance de erros de sincronização
 - Difícil manutenção

“Evolução natural” do problema produtor-consumidor

- Limitações da sincronização manual
- Precisamos controlar:
 - `wait()`
 - `notify()`
 - Semáforos
 - buffer cheio/vazio
 - exclusão mútua
- O código fica mais complexo
 - Maior chance de erros de sincronização
 - Difícil manutenção

Será que já existem estruturas prontas para resolver isso?

BlockingQueue

- Estrutura da biblioteca `java.util.concurrent`
- Implementa automaticamente:
 - Sincronização
 - espera por espaço livre
 - espera por itens disponíveis
 - comunicação entre threads
- Muito utilizada no desenvolvimento concorrente moderno

Funcionamento da BlockingQueue

- Inserção
 - `fila.put(item);`
 - Se a fila estiver cheia:
 - a thread espera automaticamente

Funcionamento da BlockingQueue

- Remoção
 - `fila.take();`
 - Se a fila estiver vazia:
 - a thread espera automaticamente

Funcionamento da BlockingQueue

- Remoção
 - `fila.take();`
 - Se a fila estiver vazia:
 - a thread espera automaticamente

A própria estrutura controla a
sincronização

Comparação das abordagens

Implementação Manual

`wait()`

`notify()`

semáforos

maior complexidade

mais propensa a erros

BlockingQueue

automático

automático

interno

menor complexidade

mais segura

Comparação das abordagens

Implementação Manual

`wait()`

`notify()`

semáforos

maior complexidade

mais propensa a erros

BlockingQueue

automático

automático

interno

menor complexidade

mais segura

Vamos ver como o mesmo problema produtor-consumidor pode ser resolvido utilizando uma BlockingQueue.

Exemplo

```
3 import java.util.concurrent.ArrayBlockingQueue;
4 import java.util.concurrent.BlockingQueue;
5
6 public class Buffer {
7
8     private BlockingQueue<Integer> fila =
9         new ArrayBlockingQueue<>(5);
10
11 public void produzir(int valor) throws InterruptedException {
12     fila.put(valor);
13
14     System.out.println("Produzido: " + valor);
15 }
16
17 public void consumir() throws InterruptedException {
18     int valor = fila.take();
19
20     System.out.println("Consumido: " + valor);
21 }
22 }
```

Exemplo

```
3 public class Produtor implements Runnable {
4
5     private Buffer buffer;
6
7     public Produtor(Buffer buffer) {
8         this.buffer = buffer;
9     }
10
11     @Override
12     public void run() {
13
14         try {
15
16             for (int i = 0; i < 10; i++) {
17
18                 buffer.produzir(i);
19
20                 Thread.sleep(500);
21             }
22
23         } catch (InterruptedException e) {
24
25             Thread.currentThread().interrupt();
26         }
27     }
28 }
```

```
3 public class Produtor implements Runnable {
4
5     private Buffer buffer;
6
7     public Produtor(Buffer buffer) {
8         this.buffer = buffer;
9     }
10
11     @Override
12     public void run() {
13
14         try {
15
16             for (int i = 0; i < 10; i++) {
17
18                 buffer.produzir(i);
19
20                 Thread.sleep(500);
21             }
22
23         } catch (InterruptedException e) {
24
25             Thread.currentThread().interrupt();
26         }
27     }
28 }
```


Exemplo

```
3 public class Consumidor implements Runnable {
4
5     private Buffer buffer;
6
7     public Consumidor(Buffer buffer) {
8         this.buffer = buffer;
9     }
10
11     @Override
12     public void run() {
13
14         try {
15
16             for (int i = 0; i < 10; i++) {
17
18                 buffer.consumir();
19
20                 Thread.sleep(1000);
21             }
22
23         } catch (InterruptedException e) {
24
25             Thread.currentThread().interrupt();
26         }
27     }
28 }
```

```
3 public class Consumidor implements Runnable {
4
5     private Buffer buffer;
6
7     public Consumidor(Buffer buffer) {
8         this.buffer = buffer;
9     }
10
11     @Override
12     public void run() {
13
14         try {
15
16             for (int i = 0; i < 10; i++) {
17
18                 buffer.consumir();
19
20                 Thread.sleep(1000);
21             }
22
23         } catch (InterruptedException e) {
24
25             Thread.currentThread().interrupt();
26         }
27     }
28 }
```

Exemplo

```
3 public class Principal {
4
5     public static void main(String[] args) {
6
7         Buffer buffer = new Buffer();
8
9         Thread produtor =
10             new Thread(new Produtor(buffer));
11
12         Thread consumidor =
13             new Thread(new Consumidor(buffer));
14
15         produtor.start();
16         consumidor.start();
17     }
18 }
```

Problemas?

- Exemplificando um problema, o tema da próxima aula..

Exemplo - Deadlock

```
3 public class Recurso {  
4  
5     private String nome;  
6  
7     public Recurso(String nome) {  
8         this.nome = nome;  
9     }  
10  
11     public String getNome() {  
12         return nome;  
13     }  
14 }
```

Exemplo - Deadlock

```
3 public class Processo1 extends Thread {
4
5     private Recurso recurso1;
6     private Recurso recurso2;
7
8     public Processo1(Recurso r1, Recurso r2) {
9         this.recurso1 = r1;
10        this.recurso2 = r2;
11    }
12
13    @Override
14    public void run() {
15
16        synchronized (recurso1) {
17
18            System.out.println(
19                "Processo 1 bloqueou " +
20                recurso1.getNome());
21
22            try {
23                Thread.sleep(1000);
24            } catch (InterruptedException e) { }
25
26            synchronized (recurso2) {
27
28                System.out.println(
29                    "Processo 1 bloqueou " +
30                    recurso2.getNome());
31            }
32        }
33    }
34 }
```

```
3 public class Processo1 extends Thread {
4
5     private Recurso recursol;
6     private Recurso recurso2;
7
8     public Processo1(Recurso r1, Recurso r2) {
9         this.recursol = r1;
10        this.recurso2 = r2;
11    }
12
13    @Override
14    public void run() {
15
16        synchronized (recursol) {
17
18            System.out.println(
19                "Processo 1 bloqueou " +
20                recursol.getNome());
21
22            try {
23                Thread.sleep(1000);
24            } catch (InterruptedException e) { }
25
26            synchronized (recurso2) {
27
28                System.out.println(
29                    "Processo 1 bloqueou " +
30                    recurso2.getNome());
31            }
32        }
33    }
34 }
```

Exemplo - Deadlock

```
3 public class Processo2 extends Thread {
4
5     private Recurso recurso1;
6     private Recurso recurso2;
7
8     public Processo2(Recurso r1, Recurso r2) {
9         this.recurso1 = r1;
10        this.recurso2 = r2;
11    }
12
13    @Override
14    public void run() {
15
16        synchronized (recurso2) {
17
18            System.out.println(
19                "Processo 2 bloqueou " +
20                recurso2.getNome());
21
22            try {
23                Thread.sleep(1000);
24            } catch (InterruptedException e) { }
25
26            synchronized (recurso1) {
27
28                System.out.println(
29                    "Processo 2 bloqueou " +
30                    recurso1.getNome());
31            }
32        }
33    }
34 }
```



```
3 public class Processo2 extends Thread {
4
5     private Recurso recurso1;
6     private Recurso recurso2;
7
8     public Processo2(Recurso r1, Recurso r2) {
9         this.recurso1 = r1;
10        this.recurso2 = r2;
11    }
12
13    @Override
14    public void run() {
15
16        synchronized (recurso2) {
17
18            System.out.println(
19                "Processo 2 bloqueou " +
20                recurso2.getNome());
21
22            try {
23                Thread.sleep(1000);
24            } catch (InterruptedException e) { }
25
26            synchronized (recurso1) {
27
28                System.out.println(
29                    "Processo 2 bloqueou " +
30                    recurso1.getNome());
31            }
32        }
33    }
34 }
```

Exemplo - Deadlock

```
3 public class Principal {  
4  
5     public static void main(String[] args) {  
6  
7         Recurso teclado =  
8             new Recurso("Teclado");  
9  
10        Recurso impressora =  
11            new Recurso("Impressora");  
12  
13        Processo1 p1 =  
14            new Processo1(teclado, impressora);  
15  
16        Processo2 p2 =  
17            new Processo2(teclado, impressora);  
18  
19        p1.start();  
20        p2.start();  
21    }  
22 }
```

Deadlock

- O que aconteceu?

Deadlock

- O que aconteceu?
 - Travou!
 - Quem está esperando quem?
 - Como evitar?

```
3 public class Processo2 extends Thread {
4
5     private Recurso recurso1;
6     private Recurso recurso2;
7
8     public Processo2(Recurso r1, Recurso r2) {
9         this.recurso1 = r1;
10        this.recurso2 = r2;
11    }
12
13    @Override
14    public void run() {
15
16        // AGORA PEGA NA MESMA ORDEM
17        synchronized (recurso1) {
18
19            System.out.println(
20                "Processo 2 bloqueou " +
21                recurso1.getNome());
22
23            try {
24                Thread.sleep(1000);
25            } catch (InterruptedException e) { }
26
27            synchronized (recurso2) {
28
29                System.out.println(
30                    "Processo 2 bloqueou " +
31                    recurso2.getNome());
32            }
33        }
34    }
35 }
```

O que mudou?

- Antes:
- Processo 1
 - Teclado → Impressora
- Processo 2
 - Impressora → Teclado

O que mudou?

- Cada processo tenta adquirir os recursos em ordem diferente
- Um processo fica esperando o recurso que está com o outro
- Surge uma espera circular
- O sistema entra em deadlock

O que mudou?

- Agora:
- Processo 1
 - Teclado → Impressora
- Processo 2
 - Teclado → Impressora

O que mudou?

- Ambos os processos seguem a mesma ordem de aquisição
- Um processo pode precisar esperar
- Porém, os recursos serão liberados normalmente
- Não existe espera circular
- O sistema continua executando

O que mudou?

- Resultado
 - Existe concorrência
 - Existe espera
 - Mas não existe deadlock

O que mudou?

- O que eliminamos?
 - Espera Circular
 - Os processos não ficam esperando uns pelos outros em ciclo
 - Todos seguem a mesma sequência para acessar recursos
 - Essa é uma forma simples de prevenir deadlock